

Section 24: Express.js with Node.js

200. What is Express?

- A JavaScript framework that allows us to create backend for websites.
- Why express?
 - Express streamlines the development process, making it faster and more efficient to build web applications compared to using raw Node.js.

201. Creating Our First Server with Express

- Remembering a backend/server:
 - The server is simply a computer. On 24x7. Always listening for a particular resource.
 - Contains an application. Application runs on server.
 - Contains a Database.
- Steps to create an express server:
 - Create directory.
 - Create index.js file.
 - Initialise NPM.
 - Install Express package.
 - Write server application in index.js.
 - Start server.
- What is localhost?
 - When we don't have server on the internet, we host locally on our device.
 - Eg. localhost:3000. Here, 3000 represents a port on the server i.e. our PC.
 - Use ``netstat -ano | findstr "LISTENING"`` to view active ports.

Example usage:

```
import express from "express";
const APP = express();
const PORT = 3000;

APP.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

202. HTTP Requests

- Stands for: HyperText Transfer Protocol
- A language that allows computers to talk to each other on the internet.
- Main requests:
 - GET: When you want to request a resource from the server. GET something from the server.

```
APP.get("/", (req, res) => {
```

```
res.send("Hello World");
});
```

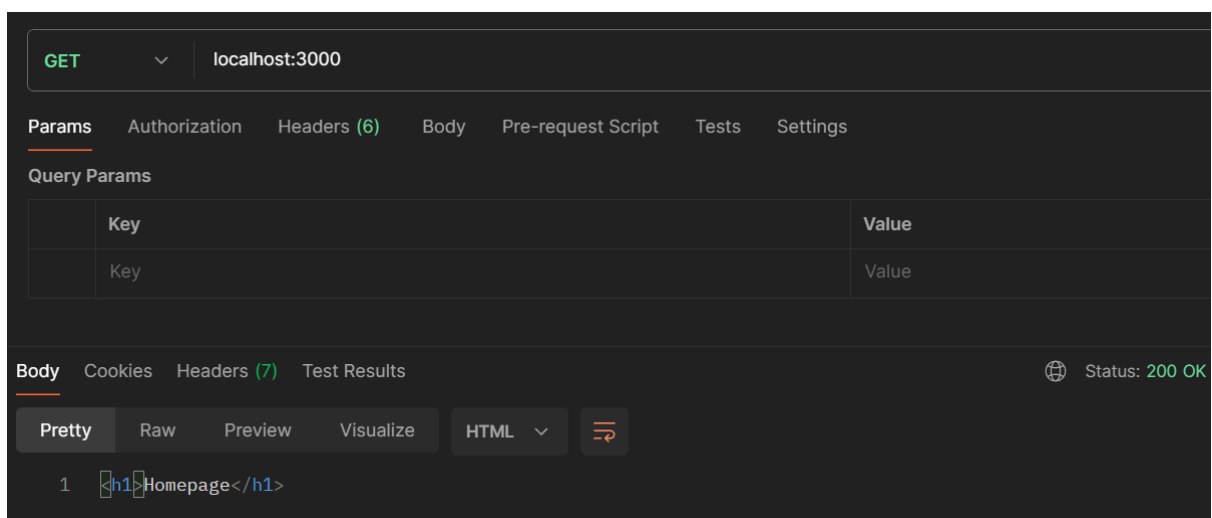
- POST: Sending a resource to the server. Eg. forms.
- PUT: Update method. When you want to replace a resource with whatever we are sending over.
- PATCH: Patch-up a resource.
- DELETE: Deletes resources from server. Client -> Server request.
- Endpoint:
 - Root: “/”.
 - Other endpoints: “/about”. Need to write GET for every endpoint.

```
APP.get("/about", (req, res) => {
  res.send("<h1>About Me</h1>");
});
```

203. Postman

- HTTP response status codes. Refer: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
 1. Informational responses (100 - 199)
 2. Successful responses (200 - 299)
 - 200: OK
 - 201: Created
 3. Redirection messages (300 - 399)
 - 301: URL changed permanently, Redirection
 4. Client error responses (400 - 499)
 - 404: Resource not found
 5. Server error responses (500 - 599)
- Use postman to develop backend without frontend.

GET:



POST:

POST localhost:3000/register?name=rishi&email=riship.dev@gmail.com

Params Authorization Headers (7) Body Pre-request Script Tests Settings

	Key	Value
☒	name	rishi
☒	email	riship.dev@gmail.com
	Key	Value

Body Cookies Headers (7) Test Results Status: 201 Created

Pretty Raw Preview Visualize Text

1 Created

PUT:

PUT localhost:3000/user/rishi

Params Authorization Headers (8) Body Pre-request Script Tests Settings

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

	Key	Value
☒	email	whatever
	Key	Value

Body Cookies Headers (7) Test Results Status: 200 OK

Pretty Raw Preview Visualize Text

1 OK

PATCH:

PATCH localhost:3000/user/rishi

Params Authorization Headers (8) Body Pre-request Script Tests Settings

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

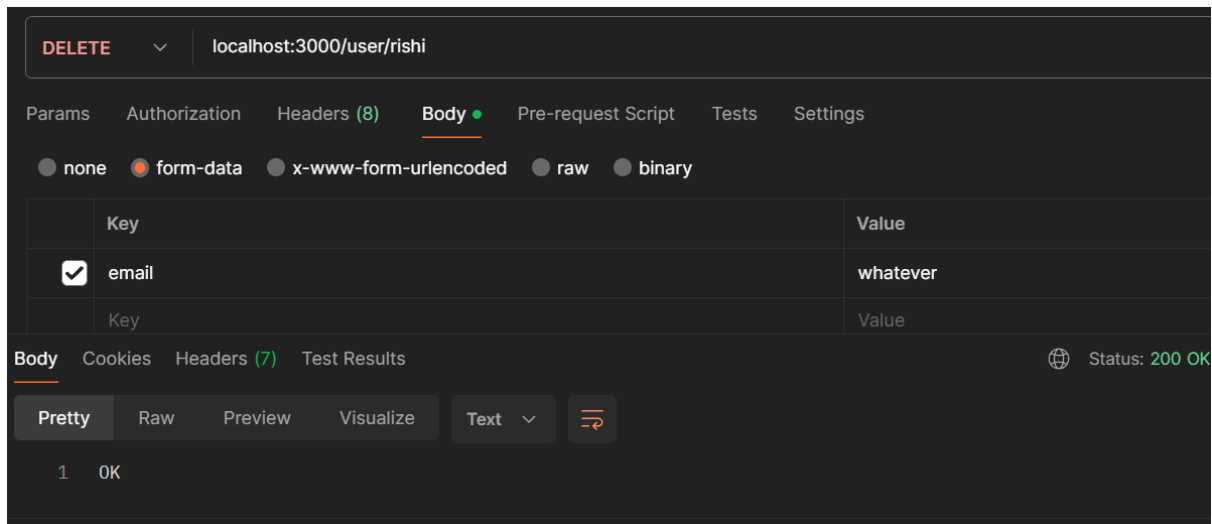
	Key	Value
☒	email	whatever
	Key	Value

Body Cookies Headers (7) Test Results Status: 200 OK

Pretty Raw Preview Visualize Text

1 OK

DELETE:



204. Introduction to middlewares

- Body-parser is the Node.js body-parsing middleware. It is responsible for parsing the incoming request bodies in a middleware before you handle it. It's commonly used in web applications built with Express.js to handle form submissions, JSON payloads, and other types of request bodies.
- Installation: `npm i body-parser;`
- Importing module: `import bodyParser from "body-parser";`
- Usually used to handle form data.

```
<form action="/login" method="POST">
  <label for="email">Email</label>
  <input type="text" name="email" required/>

  <label for="password">Password</label>
  <input type="text" name="password" required/>

  <input type="submit" value="Submit"/>
</form>
```

Email Password

- A form
 - Action – Route that the server handle this request.
 - Method – How this data to be processed in the backend.
 - Label – Labels the next input. Pairs with input element.
 - Input has two parameters:
 - Type: what kind of input e.g. text, radio button, submit (button) etc.
 - Name: A key for the data. (key: data)

```
import { dirname } from "path";
import { fileURLToPath } from "url";
```

```
const __dirname = dirname(fileURLToPath(import.meta.url));
```

- ^ Code needed to find the CURRENT file's (.js) location.
- Using body-parser

```
APP.use(bodyParser.urlencoded({extended: true}));  
APP.post("/submit", (req, res) => {  
  console.log(req.body);  
});
```

205. Custom Middlewares

- Middleware types:
 - Logging Middleware.
 - Pre-processing Middleware.
 - Authentication Middleware.
 - Error handling Middleware.
- `morgan` is a logging middleware. Used to log requests coming through server.
 - `import morgan from "morgan";`
 - `APP.use(morgan("combined"));`

```
APP.use(morgan("tiny"));
```

- In DIY MW, need to use `next()` ; at the end of function.

```
APP.use(logger);  
function logger(req, res, next) {  
  console.log(req.method);  
  console.log(req.url);  
  next();  
}
```